Auditoria técnica do `soc-final`

O repositório `LuisHenriqueMaciel/soc-final`, na revisão auditada `96dc98fbcae73ca09334d1767b902556b60f9380`, apresenta uma proposta tecnicamente relevante e bons núcleos algorítmicos isolados, mas **ainda não comprova o sistema completo descrito na proposta.** Os principais bloqueadores são: inexistência do encadeamento compressão→criptografia, equivalência prática entre os cenários PL e híbrido, reutilização insegura de nonce/IV, cobertura insuficiente do LZ4 RTL e ausência de resultados reais de síntese, implementação e medições em placa.

Parecer executivo

A plataforma escolhida é adequada: o Zynq-7000 integra processadores Cortex-A9 e lógica programável no mesmo dispositivo, permitindo comparar software, hardware e codesign de forma controlada. A Cora Z7-10 oferece dois núcleos Cortex-A9, 512 MB de DDR3, aproximadamente 2,1 Mbit de BRAM e o XC7Z010, recursos suficientes para os aceleradores propostos. [1] [2] [3]

Prioridade	Constatação	Impacto
Crítica	O multiplexador seleciona somente um entre RLE, LZ4, AES-CTR e ChaCha20	Não existe pipeline compressão→criptografia
Crítica	Cenários `SCENARIO_PL_PURE` e `SCENARIO_HYBRID` executam o mesmo código	A comparação central entre as três arquiteturas não é válida
Crítica	O “PL puro” depende do PS, de buffers DDR e de AXI DMA	Não é um datapath autônomo SPI→PL→saída
Crítica	Chaves, IVs e nonces são fixos e reutilizados	AES-CTR e ChaCha20 perdem segurança sob múltiplas mensagens
Alta	O repositório não contém projeto Vivado reproduzível, bitstream, XSA ou relatórios de implementação	Frequência, recursos e timing não podem ser auditados
Alta	O LZ4 RTL não é validado byte a byte nem decodificado por implementação independente	A compatibilidade LZ4 permanece sem comprovação
Alta	A documentação descreve janela LZ4 de quilobytes, mas o RTL usa parâmetros padrão muito menores	Resultados de compressão PS e PL não são comparáveis
Alta	O benchmark cobre combinações isoladas, não a matriz completa	Não sustenta conclusões gerais de desempenho
Média	Há 5 arquivos de testbench e 23 casos VUnit, enquanto o checklist declara 18 testbenches	Inconsistência documental
Média	Scripts e documentos contêm caminhos absolutos de uma máquina específica	O projeto não é portável nem reproduzível

Proposta do projeto

A proposta registrada no `README.md` e em `SYSTEM_ARCHITECTURE.md` pretende comparar quatro algoritmos em três arquiteturas:

Cenário	Arquitetura pretendida	Situação encontrada
PS puro	SPI→PS→DDR→algoritmos C	Parcialmente implementado e testável
PL puro	SPI→PL→compressor→cifra→buffer	Não implementado como pipeline autônomo
Híbrido	PS configura DMA e aceleradores PL	Implementado parcialmente, um acelerador por transferência
Comparação	Mesma carga nos três cenários	Benchmark atual usa cargas diferentes por algoritmo

O `pipeline_mgr.c` distingue o cenário PS de "hardware", mas não possui caminhos separados para PL puro e híbrido. Qualquer valor diferente de `SCENARIO_PS_PURE` entra no mesmo fluxo PS→DMA→acelerador→DMA→PS.

Além disso, o `axis_accel_mux.vhdl` e seu script de integração selecionam uma única rota entre os quatro aceleradores. Portanto, não é possível executar no mesmo fluxo, por exemplo:

$$\text{entrada} \rightarrow \text{LZ4} \rightarrow \text{AES-CTR} \rightarrow \text{saída}.$$

Isso é especialmente importante porque a ordem tecnicamente correta é compressão antes da cifra. Após a criptografia, a saída tende a apresentar alta entropia e deixa de oferecer redundância útil para RLE ou LZ4.

RLE

Fundamento

RLE substitui cada sequência de símbolos iguais por um par:

$$(k_i, x_i),$$

em que x_i é o byte e k_i é o número de repetições consecutivas. O formato adotado pelo projeto é `[COUNT, SYMBOL]`, com um byte para cada campo.

Para N bytes divididos em R runs, considerando a fragmentação dos runs maiores que 255, o tamanho comprimido é:

$$L_{\text{RLE}} = 2R.$$

No melhor caso, uma única sequência de N símbolos iguais gera:

$$L_{\text{melhor}} = 2\left\lceil \frac{N}{255} \right\rceil.$$

No pior caso, quando nenhum byte consecutivo se repete:

$$L_{\text{pior}} = 2N.$$

Portanto, o buffer de saída deve comportar até o dobro da entrada. A compressão somente é benéfica quando a média dos runs é superior a dois bytes.

Implementação

A implementação C em `rle.c` é direta, sem alocação dinâmica e com verificação da capacidade de saída. O núcleo `rle_engine.vhdl` desempacota o AXI4-Stream de 32 bits em bytes, acumula o run atual e empacota dois tokens de 16 bits por beat de saída.

O formato PS e PL é coerente nos casos cobertos: 64 ocorrências de `0xAA` geram `[0x40,0xAA]`, e um run de 300 bytes é fragmentado em `[255,símbolo]` e `[45,símbolo]`. O testbench `tb_rle_engine.vhdl` cobre redundância elevada, múltiplos runs, saturação em 255 e backpressure.

Lacunas

Faltam testes para:

- Entradas vazias e de um único byte.
- Runs de 254, 255, 256, 510 e 511 bytes.
- Todas as combinações válidas de `TKEEP` no último beat.
- Mudança de símbolo exatamente no limite entre dois beats AXI.
- Backpressure aleatório durante a emissão de token parcial.
- Buffer de saída insuficiente.
- Reset durante um run ativo.
- Comparação automática entre os bytes produzidos por C e RTL.
- Round-trip `RTL compress → C decompress`.

Critério de aprovação

O RLE pode ser considerado auditado quando todo stream produzido pelo RTL puder ser decodificado pela função C, incluindo entradas aleatórias, fragmentações de AXI, runs extremos e backpressure aleatório. Para qualquer entrada X, deve ser verificado:

$$D_{\mathrm{RLE}}\left(C_{\mathrm{RTL}}(X)\right) = X.$$

LZ4

Fundamento

LZ4 é um compressor baseado em LZ77. Cada sequência contém um token, literais, offset de 16 bits e comprimento de match. O token divide seus bits em dois nibbles: comprimento de literais e comprimento do match menos quatro.

A especificação oficial define offset little-endian, match mínimo de quatro bytes e condições específicas de término: a última sequência deve conter somente literais, os últimos cinco bytes

da entrada devem permanecer literais e o último match deve começar pelo menos 12 bytes antes do final. [4] [5]

A eficiência depende fortemente de:

- Tamanho da janela de histórico.
- Tamanho e função da tabela hash.
- Política de substituição.
- Profundidade da busca.
- Estratégia de seleção de matches.
- Tratamento das sequências finais.

Implementação C

A implementação `lz4_simple.c` usa uma tabela hash de 2.048 entradas com offsets de 16 bits. O compressor gera tokens compatíveis com a estrutura básica de bloco LZ4 e o descompressor trata cópias sobrepostas byte a byte.

Foram identificados três pontos de auditoria:

- Para `src_len == 4`, a expressão usada para calcular `iend - 5` forma um ponteiro antes do início do objeto, comportamento indefinido em C.
- O algoritmo permite que um match encontrado próximo do final seja prolongado até `iend`; isso precisa ser confrontado explicitamente com as regras oficiais dos cinco literais finais.
- A tabela hash é `static`, sendo inadequada para reentrância, paralelismo ou uso simultâneo por mais de um contexto.

Para uma entrada literal de 65.536 bytes, a própria codificação de extensões pode exigir aproximadamente 65.794 bytes. Um buffer de apenas `65536 + 256` bytes não é suficiente para esse caso por uma pequena margem. O benchmark atual não comprime 64 KiB com LZ4, mas a proposta declara suporte a blocos dessa dimensão.

Implementação RTL

O `lz4_engine.vhdl` usa por padrão `c_window_size = 64` e `c_hash_size = 32`. Isso diverge substancialmente da implementação C, que utiliza 2.048 posições de hash, e da documentação, que descreve janelas de 2 ou 4 KiB.

A diferença não torna o formato necessariamente inválido, mas produz algoritmos de busca incomparáveis. Uma comparação PS versus PL passa a misturar dois efeitos:

- Diferença entre processador e FPGA.
- Diferença na qualidade do compressor e no tamanho do dicionário.

Verificação

O maior problema do `tb_lz4_engine.vhdl` é que os cenários de literais, matches e dados estruturados verificam principalmente estado, comprimento de entrada e conclusão. Não há comparação abrangente dos bytes comprimidos com golden vectors nem decodificação independente do stream RTL.

A conformidade deve ser demonstrada por três oráculos:

1. `C compressor → C decompressor`.
2. `RTL compressor → C decompressor`.
3. `RTL compressor → LZ4_decompress_safe` da biblioteca oficial.

Também é necessário alimentar o descompressor do projeto com blocos produzidos pela biblioteca oficial. A especificação deixa livre a estratégia de busca do compressor, mas o decoder deve interpretar corretamente qualquer bloco válido.[4]

Critério de aprovação

O LZ4 somente deve ser anunciado como "compatível com LZ4" após:

- Corrigir o caso `src_len == 4`.
- Garantir as três regras de final de bloco.
- Dimensionar o buffer pelo limite superior documentado.
- Registrar explicitamente janela e tabela hash usadas em PS e PL.
- Capturar e verificar todos os bytes de saída RTL.
- Executar fuzzing com entradas de 0 a 65.536 bytes.
- Testar blocos malformados, offset zero, offset fora do histórico e extensões truncadas.
- Demonstrar interoperabilidade com a biblioteca oficial.

Caso essas condições não sejam atendidas, a denominação recomendada é **"compressor LZSS inspirado no formato LZ4"**.

AES-128-CTR

Fundamento

AES opera em blocos de 128 bits e admite chaves de 128, 192 ou 256 bits. No projeto é utilizada a variante AES-128.[6]

No modo CTR:

$$S_i = \mathrm{AES}_K(T_i),$$

$$C_i = P_i \oplus S_i,$$

em que T_i é o bloco de contador. A descriptografia usa a mesma operação XOR, tornando o modo apropriado a streams e blocos parciais.

O CTR definido no NIST SP 800-38A é um modo de confidencialidade, não de autenticação. Modificar um bit do ciphertext provoca alteração controlada no mesmo bit do plaintext após a descriptografia. [7]

Implementação

A implementação C em `aes_ctr.c` contém expansão da chave, dez rodadas AES-128 e incremento big-endian dos 32 bits finais do contador. O teste de software usa o vetor oficial do NIST SP 800-38A.

O `aes_ctr_engine.vhdl` possui registradores para chave, prefixo de IV e contador inicial. O `tb_aes_ctr_engine.vhdl` compara os quatro words do primeiro bloco do vetor NIST, além de testar CSR, backpressure e reset.

Riscos

O `pipeline_mgr.c` reinicializa cada operação com a mesma chave e o mesmo IV/contador. A reutilização do mesmo keystream em duas mensagens permite:

$$C_1 \oplus C_2 = P_1 \oplus P_2.$$

Esse problema é estrutural e não depende da robustez interna do AES. Chaves constantes no firmware também não constituem provisionamento seguro.

Outras lacunas são:

- Ausência de detecção de wrap do contador de 32 bits.
- Ausência de teste multibloco que valide a transição do contador.
- Ausência de testes para comprimentos de 1 a 15 bytes.
- Ausência de autenticação do ciphertext.
- Chave legível e gravável por AXI4-Lite sem controle de acesso.
- Ausência de zeroização da chave após uso ou reset.
- Implementação C baseada em S-box tabelada, potencialmente sensível a cache timing.
- Ausência de avaliação de consumo, emissões eletromagnéticas ou ataques por falha.

Recomendação

Para fins acadêmicos, o núcleo pode permanecer como AES-CTR desde que seja classificado como **primitiva de confidencialidade para benchmarking**. Para uma aplicação real, deve ser adicionado um MAC no esquema encrypt-then-MAC ou adotado AES-GCM, que fornece criptografia autenticada. [8] [9]

ChaCha20

Fundamento

ChaCha20 mantém um estado de 16 words de 32 bits:

- Quatro constantes.
- Oito words de chave.
- Um contador.
- Três words de nonce.

Cada bloco executa dez double-rounds, totalizando 20 rounds. O quarter-round utiliza apenas soma módulo 2^{32}, XOR e rotações de 16, 12, 8 e 7 bits.

A variante IETF usa chave de 256 bits, nonce de 96 bits e contador de 32 bits. O RFC exige que o nonce seja único para cada invocação com a mesma chave; a reutilização produz o mesmo keystream e revela o XOR dos plaintexts. [10]

Implementação

A implementação `chacha20.c` segue a estrutura IETF, carrega words em little-endian e executa dez double-rounds. O teste de software compara o bloco completo com o vetor da seção 2.3.2 do RFC 8439.

O `chacha20_engine.vhdl` também implementa 20 rounds e é validado com 16 words de keystream no `tb_chacha20_engine.vhdl`.

Riscos

A mesma chave e o mesmo nonce são reutilizados pelo gerenciador do pipeline. Isso é aceitável apenas para repetição controlada de vetores de benchmark; não pode ser apresentado como protocolo seguro.

Também faltam:

- Detecção de overflow do contador de 32 bits.
- Testes para 63, 64, 65, 127 e 128 bytes.
- Teste multibloco da evolução do contador.
- Zeroização dos 256 bits de chave.
- Testes de endianess cruzando firmware e RTL.
- Autenticação do ciphertext.
- Avaliação de side-channel e fault injection.

A alternativa recomendada para uso real é ChaCha20-Poly1305. O RFC 8439 define a combinação como AEAD e exige comparação de tags em tempo constante. [10]

Segurança criptográfica

O CRC-16 presente no protocolo SPI detecta erros acidentais, mas não protege contra modificações intencionais. Um atacante pode recalcular o CRC após modificar payload, flags ou contador.

O protocolo seguro mínimo deve incluir:

- Número de versão.
- Identificador do algoritmo.
- Identificador de sessão ou chave.
- Contador monotônico de mensagem.
- Nonce/IV transmitido ou derivado de forma inequívoca.
- Tamanho do plaintext e do ciphertext.
- Dados associados autenticados.
- Tag de autenticação.
- Proteção contra replay.
- Política explícita de rotação de chaves.

As chaves de teste devem ser separadas das chaves de implantação. Registradores de chave devem ser write-only, bloqueáveis após configuração e apagados em reset, timeout e erro.

Integração SPI

O projeto configura um AXI Quad SPI como escravo Mode 0, com FIFO de 256 bytes e clock externo de amostragem de 100 MHz, enquanto o ESP32 opera como mestre até 25 MHz. Essa configuração é registrada no `configure_block_design.tcl` e no firmware `spi_injector_master.c`.

Em 25 MHz e um bit por ciclo, o teto físico ideal é 3,125 MB/s, antes de preâmbulo, cabeçalho, CRC, lacunas de chip-select e software. Assim, qualquer throughput de acelerador superior a esse valor não melhora o desempenho fim a fim se a entrada continuar limitada pelo SPI.

A arquitetura documentada sugere ingestão direta pelo PL, mas a implementação real acessa o AXI Quad SPI por software e não conecta seu stream diretamente ao compressor. Logo, o cenário PL autônomo ainda exigiria:

- Parser de quadro em RTL.
- Verificação de CRC em RTL.
- FIFO ou CDC entre o domínio SPI e o domínio dos aceleradores.
- Seleção automática de compressor e cifra pelas flags do quadro.
- Encadeamento compressor→cifra.
- Framing da saída.
- Caminho de transmissão ou armazenamento sem intervenção do PS.

DMA e AXI

O projeto utiliza AXI DMA em modo simples, com canais MM2S e S2MM. Nesse modo, o software configura endereço e tamanho dos buffers diretamente; o tamanho S2MM deve ser maior ou igual ao maior pacote que poderá ser recebido, caso contrário o resultado é indefinido. [11] [12]

Isso exige atenção especial porque RLE pode duplicar o tamanho da entrada e LZ4 também pode expandir dados incompressíveis. Para cada transferência devem ser demonstrados:

- Buffer de destino dimensionado para o pior caso.
- `TLAST` emitido exatamente no último beat.
- `TKEEP` correto para 1, 2, 3 ou 4 bytes válidos.
- Invalidação e flush de cache nos pontos corretos.
- Tratamento de erros internos, slave e decode.
- Timeout sem deixar DMA ou acelerador em estado inconsistente.
- Reutilização segura após reset ou aborto.

Há ainda uma incerteza de clock. Os scripts conectam os aceleradores ao `FCLK_CLK0`, mas configuram explicitamente como 100 MHz apenas o `FCLK_CLK1` usado na amostragem SPI. Como o block design gerado não está versionado, não é possível confirmar se os contadores dos aceleradores representam ciclos de 50 ou 100 MHz.

Metodologia experimental

O `benchmark_ps.c` utiliza PMU, dez repetições, média e desvio padrão. Entretanto, cada algoritmo recebe uma carga diferente:

Algoritmo	Tamanho	Perfil atual
RLE	64 B	Alta redundância
LZ4	1 KiB	Pseudoaleatório
AES-CTR	64 KiB	Telemetria
ChaCha20	64 B e 64 KiB	Redundância/telemetria

Essa abordagem caracteriza exemplos, mas não permite comparação geral. A matriz recomendada deve aplicar todos os algoritmos e cenários aos mesmos tamanhos e perfis.

Matriz mínima

Dimensão	Valores
Cenário	PS, PL autônomo, híbrido
Tamanho	64 B, 1 KiB, 64 KiB
Perfil	Run constante, periódico estruturado, telemetria, pseudoaleatório

Dimensão	Valores
Compressor	RLE, LZ4
Cifra	AES-CTR, ChaCha20
Pipeline	Compressor sozinho, cifra sozinha, RLE→cifra, LZ4→cifra

Para os pipelines completos, isso gera pelo menos 3 cenários × 3 tamanhos × 4 perfis × 4 pipelines = 144 configurações, antes das repetições estatísticas.

Métricas

Devem ser calculadas separadamente:

$$T_{\mathrm{PS}} = \frac{\text{ciclos PMU}}{f_{\mathrm{CPU}}},$$

$$T_{\mathrm{PL}} = \frac{\text{ciclos RTL}}{f_{\mathrm{PL}}},$$

$$\mathrm{Throughput}_{\text{entrada}} = \frac{L_{\text{entrada}}}{T},$$

$$\mathrm{Throughput}_{\text{saí da}} = \frac{L_{\text{saí da}}}{T},$$

$$\mathrm{Speedup} = \frac{T_{\mathrm{PS}}}{T_{\text{acelerado}}},$$

$$CR = \frac{L_{\text{entrada}}}{L_{\text{comprimida}}}.$$

O relatório deve diferenciar latência de kernel, latência DMA e latência fim a fim com SPI. Também deve distinguir MB/s decimal de MiB/s binário.

Controle estatístico

O protocolo experimental deve utilizar:

- Pelo menos uma fase de warm-up.
- No mínimo 30 repetições por configuração.
- Média, mediana, desvio padrão, mínimo, máximo e percentil 95.
- Mesmos dados de entrada em todos os cenários.
- Política fixa para caches e interrupções.
- Medição com cache quente e cache frio em experimentos separados.
- Subtração comprovada do overhead de leitura do PMU.
- Registro da frequência efetiva de CPU e PL.
- Hash SHA-256 de bitstream, firmware e payload.

- Separação entre configuração de chave e processamento do payload.
- Registro bruto em CSV, não apenas resultados impressos na UART.

Atualmente, `pipeline_metrics_t` possui campos para ciclos, latência e throughput, mas o gerenciador deixa vários deles zerados. Portanto, a estrutura existe, porém o benchmark acelerado ainda não está completo.

Verificação disponível

A suíte contém cinco testbenches VHDL e 23 casos registrados no VUnit:

Módulo	Casos
RLE	5
LZ4	6
AES-CTR	4
ChaCha20	4
Multiplexador	4
Total	23

O checklist declara “18 testbenches”, o que não corresponde nem aos cinco arquivos nem aos 23 casos. Essa informação deve ser corrigida no `Checklist_de_Entregaveis.csv`.

As referências C de RLE, LZ4, AES-CTR e ChaCha20 possuem testes host. AES e ChaCha20 utilizam vetores oficiais; RLE e LZ4 verificam round-trip. Os drivers AXI também possuem testes, mas usam registradores e DMA simulados em memória no `test_axi_drivers.c`. Esses testes validam a lógica do driver, não comprovam funcionamento em FPGA.

A estratégia completa deve incluir:

- Teste unitário C.
- Teste unitário RTL.
- Co-simulação C/RTL.
- Teste do block design.
- Teste com DMA real.
- Teste com SPI real.
- Teste fim a fim no ESP32 e Cora Z7.
- Síntese e análise de timing.
- Captura de ILA.
- Campanha de benchmark repetível.

Reprodutibilidade

Os diretórios de Vivado e Vitis contêm somente placeholders. Os scripts esperam abrir um projeto `soc-final.xpr` já existente, mas esse arquivo não está versionado. Consequentemente, um clone limpo não consegue reconstruir o hardware apenas com o conteúdo do repositório.

Também foram encontrados caminhos absolutos `/home/luis/...` em scripts Tcl, configuração do VS Code e documentação. O próprio gate de auditoria de entregáveis rejeita esses caminhos.

Para tornar o projeto reproduzível:

1. Criar um script `create_project.tcl` que parta de um diretório vazio.
2. Usar variáveis de ambiente para Vivado, Vitis, ESP-IDF e board repository.
3. Versionar todos os parâmetros dos IPs.
4. Gerar o block design integralmente por Tcl.
5. Fixar as versões dos IPs e ferramentas.
6. Exportar relatórios de utilização, timing, DRC e potência.
7. Exportar XSA, bitstream e ELF como artefatos de release.
8. Registrar comandos, hashes e logs de build.
9. Remover links `file:///home/luis/...`.
10. Executar os testes e builds em integração contínua.

Plano de correção

Prioridade	Ação	Evidência necessária
P0	Implementar compressão→criptografia	Simulação e placa com RLE/LZ4 seguidos de AES/ChaCha
P0	Separar PL puro de híbrido	Dois datapaths e medições independentes
P0	Implementar política de nonce/IV	Teste que impeça reutilização com a mesma chave
P0	Definir segurança autenticada	AES-GCM, ChaCha20-Poly1305 ou MAC separado
P0	Corrigir e validar LZ4	Interoperabilidade com biblioteca oficial
P1	Tornar Vivado reproduzível	Clone limpo gera bitstream sem arquivos externos
P1	Completar benchmark	CSV bruto para toda a matriz experimental
P1	Executar testes em placa	Logs, UART, ILA e hashes dos binários
P1	Corrigir buffer worst-case	Testes de expansão RLE/LZ4
P1	Adicionar testes de bloco parcial	Todos os TKEEP e limites 15/16/17 e 63/64/65
P2	Implementar zeroização	Teste por leitura interna/ILA após reset
P2	Avaliar side-channel	Consumo, timing, EM e injeção de falhas
P2	Consolidar documentação	Especificação única, coerente com generics e clocks reais

Critérios finais

O projeto estará tecnicamente pronto para auditoria final quando cumprir simultaneamente:

- Clone limpo reproduz bitstream, XSA e firmware.
- Todos os testes C e RTL passam em CI.
- LZ4 RTL é decodificado pela biblioteca oficial.
- PS, PL puro e híbrido são arquiteturas realmente distintas.
- Existem pipelines RLE→AES, RLE→ChaCha, LZ4→AES e LZ4→ChaCha.
- Todos os casos são executados com dados e tamanhos idênticos.
- IVs e nonces não se repetem sob a mesma chave.
- Ciphertexts possuem autenticação ou são explicitamente classificados como não seguros.
- Resultados incluem recursos, timing, potência, latência, throughput e razão de compressão.
- Dados brutos e scripts de análise acompanham o relatório.
- Não há caminhos absolutos ou dependências não versionadas.
- As conclusões diferenciam desempenho do kernel, DMA, SPI e sistema fim a fim.

O estado atual deve ser classificado como **protótipo funcional de algoritmos e interfaces individuais**, não como demonstração concluída dos três cenários de codesign. A base de código é aproveitável, especialmente nos núcleos AES/ChaCha e no RLE, mas as alegações de pipeline completo, PL autônomo e comparação experimental ainda precisam das correções e evidências acima.

⁂

1. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Overview?contentId=GZC3RgWLSxY_qtPqsSEw_A
2. https://digilent.com/reference/programmable-logic/start
3. https://digilent.com/shop/cora-z7-zynq-7000-single-core-for-arm-fpga-soc-development/
4. https://github.com/lz4/lz4/blob/v1.9.4/doc/lz4_Block_format.md
5. https://github.com/lz4/lz4/blob/ebb370ca83af193212df4dcbadcc5d87bc0de2f0/doc/lz4_Block_format.md
6. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
7. https://csrc.nist.gov/pubs/sp/800/38/a/final
8. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-175Br1.pdf
9. https://csrc.nist.gov/pubs/sp/800/38/d/final
10. https://datatracker.ietf.org/doc/rfc8439/
11. https://docs.amd.com/r/en-US/pg021_axi_dma/Field-Descriptions
12. https://docs.amd.com/r/en-US/pg021_axi_dma/Direct-Register-Mode-Simple-DMA
13. https://digilent.com/reference/programmable-logic/guides/labview
14. https://digilent.com/reference/

15. https://csrc.nist.gov/news/2023/nist-updates-fips-197-advanced-encryption-standard
16. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.ipd.pdf
17. https://www.nist.gov/itl/773-computer-security-division-supplemental-materials
18. https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8459.pdf
19. https://csrc.nist.gov/pubs/fips/197/ipd
20. https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf
21. https://csrc.nist.gov/projects/block-cipher-techniques/bcm/current-modes
22. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf
23. https://csrc.nist.gov/pubs/fips/197/final
24. https://csrc.nist.gov/projects/block-cipher-techniques/bcm
25. https://csrc.nist.gov/pubs/fips/197/final-(1)
26. https://www.nist.gov/publications/advanced-encryption-standard-aes-0
27. https://www.nist.gov/publications/recommendation-block-cipher-modes-operation-methods-and-techniques
28. https://nvlpubs.nist.gov/nistpubs/Legacy/sp/nistspecialpublication800-38a.pdf
29. https://csrc.nist.gov/news?sortBy-lg=NewsDateTime+DESC&ipp-lg=25&topics-lg=27481|cryptography&topicsMatch-lg=ANY&page=2
30. https://link.springer.com/chapter/10.1007/978-1-4471-5607-9_3?error=cookies_not_supported&code=4f4240ca-7836-4dbd-8150-28ca3c19e1fc
31. https://link.springer.com/chapter/10.1007/978-3-642-19263-0_33
32. https://android.googlesource.com/platform/external/lz4/+/HEAD/doc/lz4_Block_format.md
33. https://developer.apple.com/documentation/compression/compression_lz4
34. https://ijneam.unimap.edu.my/images/PDF/InCAPE2021/Vol_14_SI_Dec_2021_191-197.pdf
35. https://link.springer.com/chapter/10.1007/978-3-642-15775-2_36?error=cookies_not_supported&code=83693659-f532-4285-848a-d3f22701907c
36. https://lz4.org/
37. https://docs.amd.com/r/en-US/63861-AOCL-compression/LZ4
38. https://sistemasi.ftik.unisi.ac.id/index.php/stmsi/article/view/4175
39. https://arxiv.org/ftp/arxiv/papers/1804/1804.00589.pdf
40. https://docs.zephyrproject.org/latest/develop/manifest/external/lz4.html
41. https://docs.nvidia.com/cuda/nvcomp/lz4.html
42. https://android.googlesource.com/platform/external/lz4/+/HEAD/doc/lz4_Frame_format.md
43. https://fuchsia.googlesource.com/third_party/lz4/+/refs/tags/v1.7.5/README.md
44. https://software-dl.ti.com/simplelink/esd/simplelink_msp432_sdk/3.40.00.05/docs/lz4/api_guide/html/group__lz4__api.html
45. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Performance-Monitoring-Unit
46. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM
47. https://documentation-service.arm.com/static/5e8e2ab9fd977155116a7035
48. https://documentation-service.arm.com/static/5f0377b2cafe527e86f5c247?token=

49. https://documentation-service.arm.com/static/5f0370fccafe527e86f5bfb2?token=
50. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Zynq-7000-SoC-Documents?contentId=swK4BRLdbOGDkGqgykyVUg
51. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/AXI_HP-Interfaces
52. https://www.amd.com/content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/soc/zynq-7000-product-brief.pdf
53. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Additional-Resources-and-Legal-Notices?contentId=wuTkyWqECjupE38Nkz8Wsw
54. https://docs.amd.com/r/2023.2-English/oslib_rm/Arm-Cortex-A9-Time-Functions
55. https://docs.amd.com/r/2024.2-English/oslib_rm/Arm-Cortex-A9-Time-Functions?contentId=CAwr6tkNQ3s~HOo_W2y6lA
56. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Overview
57. https://digilent.com/reference/programmable-logic/cora-z7/reference-manual
58. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Register-Summary?contentId=51DHb_rJTB80OPYz~cNJ2A
59. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Register-XDmaPs_CC_n_OFFSET_7-Details?contentId=Me_YtpSUbkShqid639UlOg
60. https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8459.ipd.pdf
61. https://rfcinfo.com/rfc-8439/4-security-considerations/
62. https://csrc.nist.gov/projects/block-cipher-techniques/BCM
63. https://owasp.deteact.com/cheat/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
64. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38c.pdf
65. https://github.com/Aayan01/OWASP-CheatSheetSeries/blob/master/cheatsheets/Cryptographic_Storage_Cheat_Sheet.md
66. https://github.com/yosang/OWASP/blob/master/cheatsheets/Cryptographic_Storage_Cheat_Sheet.md
67. https://github.com/mkrzysztofowicz/OWASP-CheatSheetSeries/blob/master/cheatsheets/Cryptographic_Storage_Cheat_Sheet.md
68. https://github.com/nokia/OWASP-CheatSheetSeries/blob/master/cheatsheets/Cryptographic_Storage_Cheat_Sheet.md
69. https://www.nist.gov/news-events/news/2022/03/announcement-proposal-revise-special-publication-800-38a
70. https://github.com/securancy/owasp-cheatsheetseries/blob/master/cheatsheets/Cryptographic_Storage_Cheat_Sheet.md
71. https://www.nist.gov/publications/recommendation-block-cipher-modes-operation-ccm-mode-authentication-and-confidentiality
72. https://github.com/YOU-i-Labs/lz4/blob/master/doc/lz4_Block_format.md
73. https://github.com/Bunco3/lz4/blob/dev/doc/lz4_Block_format.md
74. https://raw.githubusercontent.com/lz4/lz4/dev/doc/lz4_manual.html
75. https://www.nist.gov/publications/recommendation-block-cipher-modes-operation-galoiscounter-mode-gcm-and-gmac-0
76. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

77. https://csrc.nist.gov/CSRC/media/projects/cryptographic-module-validation-program/documents/security-policies/140sp4898.pdf
78. https://csrc.nist.gov/CSRC/media/projects/cryptographic-module-validation-program/documents/security-policies/140sp5032.pdf
79. https://csrc.nist.gov/pubs/sp/800/38/d/r1/iprd
80. https://csrc.nist.gov/Projects/block-cipher-techniques/news
81. https://csrc.nist.gov/news/2023/proposal-to-revise-sp-800-38d
82. https://csrc.nist.gov/pubs/sp/800/38/d/r1/2prd
83. https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html
84. https://www.avnet.com/wcm/connect/296e3765-24dc-4e63-b067-e6600c6d3356/zynq-7000-product-brief.pdf?MOD=AJPERES&ContentCache=NONE&CACHE=NONE&CVID=o8WZel0&attachment=false&id=1658846554099
85. https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000-xq.html
86. https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000-xa.html
87. https://docs.amd.com/api/khub/documents/juMnxca71Tf2gfjmNyjM8A/content
88. https://www.amd.com/content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/soc/ZynqAuto_ProdBrf.pdf
89. https://docs.amd.com/api/khub/documents/RDTkvEW6gBsFZ8SJHFw9Gw/content
90. https://docs.amd.com/r/en-US/oslib_rm/Arm-Cortex-A9-Time-Functions
91. https://github.com/Digilent/Petalinux-Cora-Z7-10/blob/master/README.md
92. https://files.digilent.com/resources/programmable-logic/cora-z7/SoV-Cora-Z7.pdf
93. https://digilent.com/blog/meet-cora-our-newest-product-line/
94. https://digilent.com/reference/programmable-logic/cora-z7/start
95. https://github.com/Digilent/digilent-xdc/blob/master/Cora-Z7-10-Master.xdc
96. https://digilent.com/shop/introductory-fpga-boards/
97. https://github.com/Digilent/Cora-Z7-10-XADC/blob/master/README.md?plain=1
98. https://digilent.com/reference/programmable-logic/zybo-z7/migration-guide
99. https://digilent.com/reference/programmable-logic/cora-z7/demos/start
100. https://digilent.com/shop/system-boards/
101. https://digilent.com/blog/let-that-zynq-in/